ASSIGNMENT 4 – CO4

NAME: K SUMANTH KUMAR REDDY
REG NO: 192425222
COURSE CODE: ITA0329
COURSE NAME: MOBILE COMPUTING

PART I – ARCHITECTURAL ANALYSIS AND DESIGN

TASK 1 – ARCHITECTURE REVIEW

The junior engineer's note is incomplete because it names broad concepts without defining their boundaries, interfaces, responsibilities or enforcement points. A useful architecture must let another engineer determine how requests travel from an application to hardware and how the OS controls resources.

Task 1.1 – Diagnosis of deficiencies

Deficiency 1: The hardware-to-software path is missing device drivers and a hardware abstraction layer. Applications do not normally control sensors, storage controllers or radio hardware directly. Drivers translate kernel requests into device-specific operations, while a HAL provides stable interfaces to higher system services. Without these elements, hardware dependencies and portability cannot be evaluated.

Deficiency 2: The kernel is described as a component that “runs applications”. The kernel does not execute application business logic. It manages processes and threads, virtual memory, scheduling, interrupts, device access, filesystems, networking and inter-process protection. User applications execute in user space and request privileged services through system calls or framework APIs.

Deficiency 3: The runtime is undefined. A runtime may load bytecode, verify classes, interpret instructions, compile hot code, manage memory and provide language services. It is not enough to say that applications are compiled and executed by it: the code format and the time of translation must be specified.

Deficiency 4: Permissions have no identified owner or enforcement point. An application declares requested capabilities, the package manager and permission controller determine grants, and framework services and kernel mechanisms enforce access. Some permissions are granted at install time, while dangerous/runtime permissions require user approval. A permission label alone does not protect a resource.

Deficiency 5: Battery saving has no trigger or mechanism. Power management responds to conditions such as screen-off, inactivity, battery-saver mode, thermal pressure and application standby state. The kernel may use CPU frequency scaling and suspend facilities; framework services schedule or defer work; applications must release wake locks and batch operations.

Deficiency 6: The application-to-OS boundary and communication path are absent. An app normally calls framework APIs; framework services validate requests and communicate with lower layers through Binder or other IPC. Direct native calls use documented system libraries and system-call interfaces. The draft does not show where requests are checked.

Deficiency 7: Native libraries and framework services are omitted. Media, graphics, database, cryptographic, location and networking facilities are typically provided through native libraries and system services. Without them, the architecture cannot explain how application features are implemented.

Deficiency 8: No isolation boundary is shown. Each application normally runs under a distinct identity or sandbox policy. Process isolation, file permissions, SELinux or platform entitlements limit cross-application access. This is essential to evaluate security.

- An engineer cannot identify which component owns hardware access.
- An engineer cannot determine whether an operation is privileged or unprivileged.
- An engineer cannot estimate startup, memory, battery or security behaviour.
- An engineer cannot define test points because interfaces and lifecycle boundaries are missing.

TASK 1.2 – REPAIRED ANDROID LAYERED ARCHITECTURE

The repaired specification below selects Android. It is a layered model from hardware to applications. Each layer identifies its responsibility, exposed interface and dependency on the layer beneath.

Layer	Responsibility	Exposes upward	Depends on below
Applications	User-facing activities, services, receivers and app logic; each app runs with an app UID and sandbox.	Application components, intents, UI and app-specific APIs.	Android framework APIs and app process/runtime.
Application Framework	Activity/task management, package management, permissions, notifications, location, window and content services.	Java/Kotlin framework classes and system-service APIs.	Binder IPC, native libraries, ART and kernel facilities.
Android Runtime (ART)	Loads DEX, verifies code, manages classes/objects, garbage collection, JIT/AOT compilation and app execution.	Runtime services, managed execution and JNI interface.	Linux process/memory services and native libraries.
Native Libraries and Daemons	Graphics, media, SQLite, TLS/crypto, libc, networking and native system services.	Native APIs and service interfaces used by framework/runtime.	HAL, drivers and kernel system calls.
Hardware Abstraction Layer	Standardises vendor-specific hardware interfaces for camera, audio, sensors and other devices.	Stable HAL interfaces to Android system components.	Kernel drivers and physical hardware.
Linux Kernel	Process/thread scheduling, virtual memory, networking, filesystems, device drivers, Binder driver, SELinux enforcement and power interfaces.	System calls, device nodes, Binder and kernel interfaces.	CPU, memory, storage, radio and peripheral hardware.
Hardware	CPU clusters, RAM, flash, display, camera, sensors, modem and battery/power circuitry.	Physical resources and interrupts.	Not applicable.

ANDROID ARCHITECTURE – SECURITY AND POWER LOCATIONS

Security enforcement is distributed rather than located in one single layer. At package installation, package management verifies package identity and signing information. The permission controller manages user-facing grants. Framework services check permissions before providing protected operations. Linux UID isolation, filesystem permissions and SELinux enforce boundaries at lower levels. Hardware-backed Android Keystore can protect cryptographic keys outside ordinary application memory.

The runtime is ART, which executes application DEX code and provides managed memory, verification and compilation services. Native code enters through JNI and documented native libraries. The kernel is the Linux kernel; it does not run application logic but schedules app processes and mediates access to protected resources.

Power management is also layered. The kernel controls scheduling, CPU frequency and low-level suspend behaviour. Android system services apply Doze, App Standby, job scheduling and background execution limits. Applications cooperate by batching network work, using WorkManager for deferrable tasks, using foreground services only for user-noticeable ongoing work and holding wake locks for the shortest possible interval.

APPLICATION REQUEST PATH

User action
↓
Application component (Activity / Service)
↓ framework API call
Framework service (permission + lifecycle + policy checks)
↓ Binder IPC / native library call
HAL or native daemon
↓ driver / system call
Linux kernel
↓
Hardware operation

Return path: hardware result → driver/HAL → service → callback/result → app

TASK 1.3 – DESIGN DEFENCE (UNDER 100 WORDS)

The repaired description is an architectural specification because it identifies ordered layers, assigns each layer a bounded responsibility, defines the interface exposed upward and names the dependency below. It locates ART, the Linux kernel, permission checks, SELinux and power-management mechanisms explicitly. The request path shows how an application reaches protected hardware through framework services, IPC, HAL and drivers. These details can be tested and implemented. A marketing overview would describe benefits such as speed or security without defining components, enforcement points, dependencies or observable behaviour.

TASK 2 – PLATFORM REQUIREMENT AND COMPARISON

Scenario: a secure messaging application must receive messages in the background on Android, iOS and Windows. The comparison focuses on the architectural facts that affect message delivery, isolation, responsiveness and energy use.

Task 2.1 – Problem model

Input: platform architecture information for Android, iOS and Windows, including kernel family, process model, runtime, application framework, IPC, background execution policy and security boundary.

Output: a decision matrix and a platform implementation strategy showing how incoming messages are delivered when the screen is off and what happens when an app is suspended or terminated.

Objective: provide reliable, secure and timely message delivery without keeping the CPU or radio continuously active.

Constraints: background execution is restricted; messages must be authenticated and encrypted; the application must conserve battery; callbacks must not block the UI; network connectivity may be intermittent; user privacy and notification preferences must be respected.

Decision: use each platform's supported push-notification service for wake-up/notification delivery, then retrieve message content securely when the OS grants execution time. Do not rely on a permanent background socket as the sole delivery mechanism.

Required information: kernel/process model, app sandbox, runtime model, background-task limits, push service, IPC and notification APIs, data protection and lifecycle restoration.

TASK 2.2 – HYPOTHESIS

The background-execution and process-lifecycle policy will affect message delivery more than the kernel family alone, because all three platforms can receive network events at the system level but differ in how long a third-party application may execute while backgrounded or terminated.

TASK 2.3 – COMPARISON MATRIX

Dimension	Android	iOS	Windows	Messaging consequence
Kernel family	Linux kernel with a monolithic modular design; kernel modules and drivers provide device support.	Darwin foundation with XNU hybrid kernel combining Mach and BSD components.	Windows NT kernel family with a hybrid design and executive subsystems.	Kernel family affects drivers, scheduling and IPC, but app background policy is the decisive delivery constraint.
Kernel interface	Linux system calls, Binder driver, device nodes and proc/sys interfaces; app code normally uses framework APIs.	Mach traps/ports and BSD/POSIX-derived interfaces are mediated by frameworks and entitlements.	Win32/NT system interfaces; packaged apps commonly use WinRT APIs.	Apps should use documented interfaces rather than depend on private kernel interfaces.
Runtime	ART executes DEX; verification, JIT and profile-guided AOT compilation; managed heap and GC.	Objective-C runtime supports dynamic dispatch, metadata and messaging; Swift uses native compilation and ARC.	WinRT is an API/runtime platform, not simply a single managed VM; apps may use .NET, C++ or JavaScript runtimes.	Runtime affects startup and memory behaviour; background delivery still depends on OS scheduling and push services.
Framework	Android SDK: Activity, Service, BroadcastReceiver, JobScheduler, WorkManager and notification APIs.	Cocoa Touch/UIKit or SwiftUI, UserNotifications, BackgroundTasks and app lifecycle APIs.	UWP/Windows App SDK APIs, background tasks and notification services, depending on app model.	Use platform-specific notification and background-task contracts.
Isolation	Per-app Linux UID, sandbox, SELinux and permission checks.	Per-app sandbox, code signing, entitlements and data-container separation.	AppContainer/package identity and capability declarations for packaged apps.	Separate local message stores and restrict access to tokens and attachments.
IPC	Binder is primary framework IPC;	XPC and Mach ports, plus	COM/WinRT activation and app	Use OS-supported IPC

	intents and content providers are higher-level patterns.	framework-level callbacks and URL schemes.	services; named pipes and other mechanisms where permitted.	and validate caller identity and data.
Background delivery	Firebase Cloud Messaging or other push transport wakes/alerts app subject to priority and background limits.	Apple Push Notification service delivers notifications; background notifications are opportunistic and system-controlled.	Windows Push Notification Services and background activation depend on app model and policy.	Never promise unlimited execution; design idempotent fetch and state restoration.

TASK 2.4 – SCREEN-OFF MESSAGE TRACE

Android trace (illustrative sequence):

1. The modem receives network data and the radio/network stack reports packets to the kernel.
2. The Linux networking subsystem and driver process the packet; the device may be in a low-power state.
3. Google Play services or the configured push transport receives the push event and the OS decides whether the app can be awakened.
4. The framework delivers a permitted callback or notification event to the application process.
5. ART resumes or starts the process and dispatches the managed callback.
6. The app validates message metadata, fetches encrypted content if required, writes it to private storage and posts a notification.

iOS trace (illustrative sequence):

7. The cellular/Wi-Fi subsystem receives the push notification through the system networking path.
8. Apple Push Notification service routes the notification to the device; system services validate and handle it.
9. The OS decides whether to display an alert or grant a limited background notification execution opportunity.
10. The application delegate or notification service extension receives the appropriate callback when allowed.
11. The app uses its runtime and framework APIs to decrypt permitted content, update local state and schedule follow-up work.

TASK 2.5 – CHALLENGE: APP TERMINATED BY THE OS

Changed condition: the OS has terminated the application because of memory pressure. Prediction: the application process will not receive an ordinary in-process callback while it is absent. A platform push service may still display a notification or launch the app under permitted conditions, but silent/background delivery is not guaranteed. The responsible elements are the process manager, push notification service and background execution policy.

Android relevant trace: push service receives the event → system checks message priority and app restrictions → notification may be displayed or app process may be started for a permitted callback → ART initialises the process → app restores state from durable storage. If the user force-stopped the app, delivery behaviour is more restricted until the app is launched again.

iOS relevant trace: APNs delivers the notification → iOS decides whether to show it or grant a background opportunity → if permitted, app is launched and receives a notification callback;

otherwise no background code runs and the visible alert may still appear. Therefore, the prediction is correct: process termination changes callback availability, and the application must treat the server as the durable source of truth.

TASK 3 – RUNTIME ENVIRONMENT ANALYSIS

Runtime analysis distinguishes the code package delivered to the device, the component that loads it, the point of machine-code generation and the memory-management model. The three platform labels in the assignment are historical shorthand and should not be interpreted as identical runtime architectures.

Task 3.1 – Execution models

Android: application code is commonly packaged as DEX bytecode inside APK/AAB-derived installation artifacts. The package manager installs the app; ART loads and verifies DEX. ART may interpret or JIT-compile methods during execution and may use profile-guided AOT compilation for frequently used code. Native libraries are already machine-code binaries for the target ABI.

iOS: applications are generally delivered as signed native executable code compiled ahead of time for supported architectures. The dynamic loader maps executable and framework images, while the Objective-C runtime performs class registration, method lookup and dynamic dispatch. Swift code is also generally compiled to native machine code; ARC manages object lifetime.

Windows: packaged applications may use native machine code, .NET intermediate language or other supported forms depending on the framework and language. The loader and relevant runtime (for example CLR for .NET) load and execute the application. WinRT is a set of modern Windows APIs and type system, not a single universal bytecode virtual machine.

TASK 3.2 – ANDROID COLD-LAUNCH SEQUENCE

Step	Operation	Component	Precondition / purpose
1	User taps icon; launcher resolves package/activity.	Launcher and Package Manager	Installed package and launcher entry exist.
2	System checks process state and launch intent.	ActivityTaskManager / ActivityManager	Package metadata and task policy are available.
3	If process absent, system requests app process from Zygote.	Zygote / system process	Zygote is running and process resources are available.
4	App process is forked and app UID/security context applied.	Linux kernel + Zygote	Kernel process, memory and credential services are active.
5	ART initialises runtime and loads app classes/DEX.	ART	Process exists and runtime libraries are mapped.
6	Application object and first Activity are created.	Android framework	Main thread/Looper and app component metadata exist.
7	Activity lifecycle callbacks run; resources and views are loaded.	Framework + app code	Runtime and framework initialisation completed.
8	Layout, measure, draw and frame composition	View system, RenderThread,	Window and display services are ready.

	occur.	SurfaceFlinger	
9	First frame is presented.	Display compositor / hardware	A rendered buffer is available.

Each step is dependent on the previous one. For example, application callbacks cannot run before the process and main thread exist; view rendering cannot complete before resources and window services are available. The sequence also gives measurable points for cold-start profiling.

TASK 3.3 – RUNTIME STRATEGY COMPARISON

Topic	**Android ART**	**iOS Objective-C / Swift**	**Windows runtime choices**
Interpretation	ART can interpret DEX instructions before or alongside compilation.	Native machine code is executed; Objective-C runtime provides dynamic messaging and metadata.	Native code executes directly; .NET IL is JIT/AOT compiled by CLR depending on deployment.
JIT	ART can JIT hot methods during execution.	Traditional iOS app execution is not based on a general third-party-app JIT model; platform restrictions apply.	CLR may JIT IL; Native AOT can produce native code.
AOT	Profile-guided AOT can compile selected code based on profiles; installation/runtime policy varies by version.	Native compilation occurs before delivery for supported targets.	Native binaries or AOT-published .NET apps are possible.
Memory	Managed heap with tracing garbage collection; JNI/native allocations need separate care.	Swift/Objective-C ARC uses reference counting with compiler/runtime support; cycles require weak/unowned references or cycle-breaking.	Managed .NET uses GC; native apps use explicit/RAII patterns.
Native access	JNI and NDK libraries connect managed code to native APIs.	Objective-C/Swift call system frameworks and C APIs directly.	WinRT projections, P/Invoke and native APIs connect app code to OS services.
Responsiveness	GC pauses and allocation pressure can affect latency; ART uses concurrent/optimised GC strategies.	ARC retain/release work is distributed through object operations; retain cycles can leak memory.	GC pauses can occur in managed apps; native code has different allocation risks.
Portability	DEX and framework model support a broad	Binaries target Apple-supported	Portability depends on language/runtime and

	Android device ecosystem; native ABI dependencies reduce portability.	architectures and APIs; ecosystem is tightly controlled.	Windows API version.

Interpretation means executing instructions without first converting every method to native machine code. JIT compilation converts selected code during execution; AOT compilation converts code before execution or installation. ART uses a combination of interpretation, JIT and profile-guided AOT. iOS uses native compilation for app code, with Objective-C runtime dynamic dispatch and ARC; Objective-C runtime is not a Dalvik-style bytecode interpreter. Windows varies by app language and deployment model.

MEMORY MANAGEMENT, NATIVE INTERFACES AND LOW-MEMORY DECISION

Garbage collection reclaims unreachable managed objects by tracing references from roots. It reduces manual memory-release errors but introduces allocation and collection work. A collection may pause application threads or consume CPU, although modern collectors use concurrent and generational techniques to reduce visible pauses. ARC inserts retain/release operations so object lifetime is adjusted as references are created and removed. ARC avoids a general tracing collection for ordinary reference-counted objects, but cyclic references can keep objects alive indefinitely.

Android reaches native code through JNI or NDK APIs. JNI transitions must be controlled because excessive crossings and unmanaged allocations can increase latency and complicate memory ownership. iOS code reaches native OS facilities through system frameworks and C/Objective-C interfaces. Windows applications use WinRT projections, .NET interop or native Win32 interfaces.

For a low-memory device, a carefully designed native/ARC model can offer predictable object-release timing, but it is not automatically superior: cycles, allocation churn and framework overhead still matter. ART is suitable where the Android ecosystem and managed safety are priorities, provided the app limits object churn and caches. The final choice should be based on measured peak memory, startup and latency on the target device. If the target is a very constrained device, a small native implementation with bounded allocations may be preferable to any large general-purpose framework.

TASK 3.4 – COLD START VS WARM START

Launch stage	**Cold start**	**Warm start**	**Reason**
Process creation	Required	Skipped if process alive	Cold start needs a new process.
ART/runtime initialisation	Required	Already initialised	Runtime state remains in process memory.
Class loading	Many classes loaded	Most needed classes may already be loaded	Class metadata may remain cached.
Application object	Created	Usually reused	Application instance survives while process survives.
Activity creation	Required	May be recreated or resumed depending on task state	Activity lifecycle depends on navigation and process state.
First frame	Rendered	Rendered after resume/update	A visible frame must still be produced.

TASK 3.5 – DEPENDENCY ANALYSIS

Kernel process and memory services must exist before the application process can be created because the process requires an address space, credentials, scheduling and mapped memory. Runtime initialisation must precede managed application code because classes, heaps, exception handling and the main thread depend on runtime services. Framework callbacks occur only after the framework has attached the application and created the relevant component. This dependency ordering prevents callbacks from referring to unavailable runtime or process state.

TASK 4 – SECURITY FORENSICS: SECURENOTES

The original SecureNotes design contains four independent weaknesses. The repairs below use Android as the selected platform and preserve the application's legitimate behaviour while reducing unnecessary privileges and protecting secrets.

Original decision	**Mechanism bypassed**	**Why OS protection is insufficient**
Requests every permission at install for future use	Least-privilege permission model and runtime permission prompts	A user may grant unnecessary access; broad grants increase the impact of compromise and reduce informed consent.
Stores login token in a file readable by other apps	Application sandbox, UID isolation and secure storage	If the file is exposed through a shared location or weak permissions, another app may copy the bearer token and impersonate the user.
Accepts updates from any source without producer verification	Package signing and update identity verification	An attacker can distribute a modified package or downgrade/replace trusted code if the installation path is not verified.
Uses plain unencrypted network connections	TLS transport protection and server authentication	A network attacker can read or modify credentials and notes in transit; the OS cannot add encryption to an application protocol that deliberately uses plaintext.

TASK 4.2 – ATTACK SCENARIO

Scenario: a user installs SecureNotes and grants broad permissions. The application stores a bearer token in a shared or world-readable file and sends requests over HTTP. A malicious application on the same device scans accessible shared storage or obtains the exposed file through an unsafe export. It copies the token and sends it to the attacker's server. Separately, an attacker controlling a public Wi-Fi access point observes plaintext traffic and can alter an HTTP response.

12. The user signs in; SecureNotes receives a bearer token.
13. The app writes the token to a location that is readable outside its private sandbox. Confidentiality is first lost at this write/read boundary when the malicious app can access the token.
14. The malicious app copies the token and reuses it against the service. Account integrity is lost when the server accepts the stolen token as valid.
15. During a later HTTP session, a network attacker observes note contents and may modify a response. Confidentiality and integrity in transit are lost at the unencrypted network boundary.
16. If a modified update is accepted without signature/identity verification, malicious code may replace the legitimate application and persist the compromise.

TASK 4.3 – MINIMAL REPAIRS

Decision to repair	Android mechanism and implementation	Result
Permission requests	Declare only required permissions; request dangerous permissions at the point of use; explain purpose and handle denial.	User grants only the capabilities needed for the selected feature.
Token storage	Store token in app-private internal storage; encrypt sensitive data using a key protected by Android Keystore; avoid exported providers and shared files.	Other ordinary apps cannot read the private token file; key material is protected by Keystore.
Updates	Distribute through a trusted channel and preserve package signing identity; verify release pipeline and signing key custody.	Android package verification and signing continuity help prevent unauthorised replacement.
Network	Use HTTPS with modern TLS, hostname validation and certificate validation; consider pinning only with a safe rotation plan.	Credentials and note content are encrypted and server identity is checked.

TASK 4.4 – REGRESSION CHECK

Legitimate behaviour: the user may choose to attach a photograph to a note. The repair does not remove camera functionality. The app requests camera permission only when the user selects the camera feature. If permission is granted, the framework camera API provides access and the app stores the resulting image in private app storage. If permission is denied, the app continues to support text notes and displays a clear explanation. The repair therefore preserves the agreed feature without requesting unrelated permissions.

TASK 5 – POWER MANAGEMENT REASONING

The fitness application currently holds the processor awake permanently. This prevents deep idle states and causes unnecessary CPU and radio energy use. A state-based policy should keep the device awake only while a useful foreground operation or bounded sensor event is being processed.

State	Hardware/kernel behaviour	Entry condition	Exit condition	Allowed work
ACTIVE	Display on; CPU may use performance cores; sensors and radios available as needed; scheduler runs foreground tasks.	User opens workout screen or an active workout requires immediate processing.	Screen off/inactivity, user pauses workout or thermal/battery restriction.	UI, sensor sampling, short processing and bounded network transfer.
IDLE	Display off; CPU frequency reduced; background work restricted; sensor batching enabled where supported.	No user interaction and no immediate interaction requirement.	Motion/sensor event, scheduled job, push event or user interaction.	Low-rate batched sensors, permitted system jobs and short callbacks.
SUSPENDED	CPU enters low-power suspend; most app threads stop; wake sources remain configured.	No pending high-priority work and system policy allows suspend.	Hardware wake event, alarm, push/network event or user input.	No continuous app execution; only system-controlled wake handling.
BATTERY_SAVER	CPU performance and background scheduling constrained; display/network policy may be reduced.	Battery reaches configured threshold or user enables saver.	Charging or user disables saver / battery threshold policy changes.	Essential tracking at reduced rate; deferred uploads and restricted nonessential work.

TASK 5.2 – URGENT PUSH WHILE SUSPENDED

The unsafe assumption is that the application must keep the processor awake to receive every urgent notification. A suspended device cannot guarantee that arbitrary application code continues running. A push-based wake-up mechanism lets the OS or push service receive the event and grant a bounded execution opportunity or display a notification. A partial wake lock may be acquired only after the system has delivered a permitted event and only for the short operation that genuinely requires CPU execution.

- Stop holding a permanent partial wake lock.
- Use push notifications for urgent server events and use high-priority delivery only for genuinely time-critical, user-visible cases.
- Persist work before releasing execution time; make processing idempotent so retries do not duplicate records.
- Use WorkManager/JobScheduler for deferrable uploads and sensor batching for routine measurements.
- Do not assume every silent push will wake an app immediately; design server-side retry and later reconciliation.

TASK 5.3 – BATTERY SAVER AT 15%

Layer	**Change when battery saver activates**	**Expected effect**
Kernel	Prefer lower CPU frequencies/efficiency cores where supported; reduce nonessential wakeups and permit deeper idle states.	Lower peak performance and lower energy use.
Runtime	Reduce unnecessary background execution and defer non-urgent managed work; avoid allocation-heavy polling loops.	Less CPU activity and fewer runtime/GC wakeups.
Application	Increase sensor sampling interval, batch records, defer uploads to Wi-Fi/charging where acceptable, stop animations and nonessential refresh.	Reduced tracking precision or delayed updates, but essential workout records remain available.

User-visible trade-off: location or activity updates may appear less frequently while battery saver is active. This is acceptable when the user has selected a battery-saving mode and the application clearly indicates reduced update frequency. Emergency or safety-critical events can be assigned a separate priority path.

TASK 6 – RURALCARE FIELD-DATA COLLECTION DESIGN

RuralCare must support modern Android phones, iPhones and legacy feature phones that have only J2ME/JME or WML capability. Workers may be offline for hours, need a ten-hour shift and must protect patient data if a device is lost.

Stage 1 – Problem abstraction

Input: device class, patient data fields, worker identity, battery level, storage capacity, network state, permissions and sync status.

Output: a secure field-data collection service that stores records offline and synchronises them exactly once at the logical level when connectivity returns.

Objective: maintain data confidentiality, integrity and availability across heterogeneous devices while completing a ten-hour shift.

Constraints: offline periods up to several hours; different runtimes and storage capabilities; limited legacy-phone security; battery limitations; intermittent connectivity; no data loss or duplication; lost-device risk; variable network cost.

Decisions: whether to build native apps for modern platforms, provide a restricted legacy interface, what data may be stored locally, how to authenticate workers, how to encrypt records and how to make synchronisation idempotent.

STAGE 2 – PLATFORM STRATEGY

Recommended strategy: use native Android and iOS applications for Classes A and B, and a restricted WML/J2ME-compatible workflow for Class C that stores the minimum possible data locally. The system should share a common server API and data model rather than forcing one runtime across all devices.

- Class A Android: native Kotlin/Java application using Android framework, ART, app-private database, Android Keystore and WorkManager/foreground service only for justified active tracking.
- Class B iPhone: native Swift application using UIKit/SwiftUI, system frameworks, Keychain and background task/push facilities within iOS policy.
- Class C J2ME: MIDlet where supported, with constrained local record storage and compact encrypted payloads if the device supports a suitable cryptographic library and key handling. If trustworthy encryption is unavailable, avoid persistent sensitive patient details and use short-lived server sessions.
- Class C WML: provide a minimal server-rendered workflow over a protected connection where the browser supports it. Keep sensitive records server-side and minimise cached pages.

A single cross-platform codebase is not selected because legacy devices do not share modern runtime capabilities or security services. A common backend contract and common validation rules provide consistency while native clients use the strongest platform-specific protection available.

STAGE 3 – DEVICE CONSTRAINT TABLE

Device class	Runtime	Security mechanisms	Power controls	Offline storage	Connectivity
A – Android	ART; Kotlin/Java and optional native libraries.	App UID sandbox, SELinux, runtime permissions, Keystore, TLS, verified package signing.	Doze, App Standby, WorkManager, foreground service, sensor batching, wake locks.	Encrypted SQLite/Room or encrypted files in app-private storage; unique record IDs and sync journal.	Wi-Fi, cellular, intermittent or no network.
B – iPhone	Native Swift/Objective-C runtime; ARC.	Sandbox, code signing, entitlements, Keychain, Data Protection, TLS.	System background task limits, push, location modes and Low Power Mode.	Protected local database/files with per-record IDs and sync journal.	Wi-Fi/cellular; background transfer subject to OS rules.
C – J2ME	Java ME/MIDP runtime on supported feature phones.	Device-dependent application permissions and limited secure storage; capability varies by model.	Small CPU/battery; MIDlet lifecycle and device-specific idle policy.	RecordStore or minimal encrypted payload if supported; otherwise no long-lived sensitive cache.	GPRS/2G or other feature-phone data connection; often unreliable.
C – WML	Embedded WAP browser; server-rendered WML pages.	Browser/device dependent TLS and limited local security controls.	Browser session and device standby controls.	Prefer no local patient database; use server-side session and minimal cache.	WAP gateway or supported mobile data transport.

INFORMATION RETAINED FOR SAFE SYNCHRONISATION

Field	Purpose
Record UUID	Globally unique identifier prevents the same field record being inserted twice.
Device ID / installation ID	Identifies the originating client without using a personal hardware identifier as an authentication secret.
Worker ID	Associates the record with an authenticated field worker.
Patient reference	Links the entry to the correct patient while minimising unnecessary personal data.
Created-at timestamp and timezone	Supports ordering, audit and conflict analysis when devices have different local clocks.
Local sequence number	Preserves order within one device even if the wall clock is incorrect.
Schema version	Allows the server to validate and migrate records created by older clients.
Payload hash / integrity tag	Detects accidental corruption or tampering during storage and transfer.
Sync status and attempt count	Distinguishes queued, in-progress, acknowledged and failed records.
Server acknowledgement ID	Proves that the server accepted the record and prevents duplicate retries.
Conflict version / revision	Supports deterministic conflict resolution when the same record changes on multiple devices.

A record is not deleted merely because a request was sent. It is marked synchronised only after the server returns a durable acknowledgement containing the record UUID or acknowledgement ID. If the response is lost, the client retries the same UUID; the server's idempotency rule returns the original result instead of creating a duplicate.

STAGE 4 – COMPLETE SOLUTION SPECIFICATION

4.1 Common logical architecture

Field worker
↓
Presentation layer (forms, validation, status)
↓
Domain layer (record creation, consent, sync state machine)
↓
Data layer (encrypted local queue, metadata, audit log)
↓
Platform services (Keystore / Keychain / MIDP permissions)
↓ TLS + authenticated API
RuralCare API gateway
↓
Authentication + validation + idempotency service
↓
Patient record database + audit log + backup

The presentation layer runs on the main/UI thread and performs only short operations. Validation and encryption run on a background executor. Local database transactions are atomic. Synchronisation runs through platform-approved background work. A foreground service is used only during an explicitly active, user-visible collection session when continuous operation is justified.

4.2 Data capture and offline storage

17. Authenticate the worker and obtain a short-lived access token; store refresh credentials in platform secure storage where available.
18. Load the minimum patient reference information required for the current visit.
19. Validate required fields on-device, including patient reference, form version, timestamp and consent state.
20. Generate a random UUID for the record and a monotonically increasing local sequence number.
21. Encrypt the record payload before writing it to persistent local storage. Bind metadata to the ciphertext using authenticated encryption where supported.
22. Commit the encrypted payload and sync state in one database transaction.
23. Display a saved-offline indicator only after the transaction succeeds.
24. Do not store plaintext patient data in logs, notifications, crash reports or shared external storage.

4.3 Security and lost-device behaviour

- Android: use Keystore-protected keys, encrypted private storage, app sandbox and screen-lock/device policy where managed.
- iOS: use Keychain for secrets and Data Protection classes for sensitive files; enforce application authentication where required.
- Legacy devices: keep sensitive data server-side whenever possible; restrict the local cache to non-sensitive references or short-lived encrypted records only if cryptographic capability is verified.
- All network communication uses TLS with server authentication; the API requires authenticated workers and validates authorisation for every patient record.
- Lost device: revoke the device session, expire refresh tokens, block the installation ID, and issue remote wipe through enterprise management where available.
- Use least-privilege access and separate worker roles; log access and synchronisation events without logging patient payloads.

STAGE 4 – SYNCHRONISATION AND FAILURE SPECIFICATION

4.4 Ordered synchronisation procedure

25. Connectivity monitor reports a usable network; the client checks that the worker session is still valid.
26. Client selects a bounded batch of unsynchronised records ordered by local sequence number.
27. Client verifies local integrity tags and decrypts only in memory for request construction.
28. Client sends each record with its UUID as an idempotency key, schema version and authenticated user/device context.
29. Server authenticates the worker, checks authorisation, validates schema and payload, and checks whether the UUID has already been accepted.
30. If UUID is new, server commits the record and audit entry in one transaction. If UUID already exists, server returns the original acknowledgement.
31. Client receives a durable acknowledgement and marks only the acknowledged UUID as synchronised.
32. If the connection breaks before acknowledgement, client leaves the record queued and retries later using the same UUID.
33. After a complete batch, client records last successful sync time and displays the number of remaining queued records.

4.5 Interrupted synchronisation

The client uses at-least-once transmission with server-side idempotency to achieve exactly-once logical insertion. Network requests may be repeated, but the server creates only one logical record for each UUID. The client never deletes a queued record before receiving an acknowledgement. A partial batch can therefore resume safely after an app crash, phone restart or network loss.

Failure	Client action	Server/client recovery
No network	Keep encrypted queue; retry with backoff.	No record is lost; sync resumes on connectivity.
Server timeout after commit	Retry same UUID.	Server returns existing acknowledgement; no duplicate.
App process killed	Recover queued records from durable DB on next launch.	Sync state survives process death.
Invalid schema	Mark record as needs-review; do not discard.	Server returns validation error and supported schema.
Expired token	Refresh or request worker sign-in.	No unauthenticated upload is accepted.
Device lost before sync	Revoke device credentials; rely on encrypted local storage and management wipe.	Unsynchronised records may be unrecoverable if device cannot be recovered; this residual risk must be disclosed.

STAGE 5 – DESK-CHECK: CLASS A OFFLINE SIX HOURS, BATTERY 20%, DEVICE LOST

Step	State / event	Decision and protective action	Outcome
1	Worker signs in with 100% battery.	Short-lived token stored securely; encrypted local queue initialised.	No patient record exposed.
2	Network becomes unavailable.	App enters OFFLINE state; new records get UUIDs and local sequence numbers.	Records are encrypted and queued.
3	Six hours pass; multiple records collected.	Sampling and UI remain bounded; no repeated network polling.	Records remain locally durable.
4	Battery reaches 20%.	Battery-aware policy reduces nonessential refresh and defers sync until a suitable network/charging opportunity.	Essential capture continues; nonessential work reduced.
5	Device is lost before sync.	Administrator revokes installation/session credentials and requests remote wipe if device is managed and reconnects.	Remote wipe may be delayed while offline.
6	Attacker attempts to inspect storage.	Android app sandbox and Keystore-backed encryption protect the local queue on a locked, non-rooted device.	Ciphertext may be visible but patient payload should remain unreadable.
7	Recovery decision.	If device is recovered, authenticate and sync queued records; if permanently lost, unsynchronised records may be lost.	No server copy exists for records never uploaded.

Exposure: on a correctly configured locked Android device, raw encrypted data may be present but plaintext patient content and cryptographic keys should not be readily accessible to another ordinary application. Data lost: records that were collected only on the lost device and never synchronised may be permanently unavailable if the device cannot be recovered. Recoverable: records already synchronised before the offline period remain on the server; unsynchronised records are recoverable only if the device is recovered or a secure backup exists.

STAGE 6 – ADVERSARIAL TESTS AND REFINEMENT

Adverse condition	Why challenging	Prediction	Result / refinement
Stolen unlocked phone	Attacker may use the active app session and view records in memory.	Encryption at rest alone may not protect an unlocked session.	Require short inactivity lock, app re-authentication for sensitive actions, managed-device policy and remote revocation.
Sync interrupted after server commit	Client may not receive acknowledgement and may retry.	Same UUID is resent; server idempotency returns existing acknowledgement.	No duplicate logical record; retain queue until acknowledgement.
Class C phone lacks secure storage	Persistent plaintext records may be readable if device is lost.	Legacy device cannot meet the same at-rest protection guarantee.	Do not persist sensitive details; use minimal server-side workflow or exclude that device from high-sensitivity data capture.
Device clock is incorrect	Records may appear out of order or conflict.	Local sequence and server receive time preserve deterministic ordering.	Store both device time and server time; flag clock anomalies.
Storage becomes full	A record may fail to commit after form entry.	App must not show success until atomic write completes.	Warn worker, stop accepting new record or use approved fallback; never silently discard.
Token expires while offline	Worker cannot refresh credentials without network.	Permit encrypted offline capture under policy, then re-authenticate before upload.	Use short-lived access tokens and protected refresh credentials; apply offline access expiry rules.

Critical decisions: encrypt before persistence, assign stable UUIDs, use server-side idempotency and acknowledge only durable commits. The main failed assumption is that every legacy device offers modern secure storage. The design is refined by reducing sensitive local storage on Class C and applying stronger access controls to unlocked modern devices.

PART II – TASK 1: ANDROID VS iOS OS MODEL AND LIFECYCLE VALIDATION

The practical investigation uses Android as the observable platform and compares its architecture with the documented iOS model. Android evidence can be collected through adb shell, dumpsys, ps, /proc and application lifecycle logging.

1.1 Layer-by-layer responsibility mapping

Layer	Android	iOS
Kernel/core OS	Linux kernel: scheduler, memory, drivers, networking, Binder driver and SELinux.	Darwin/XNU: Mach and BSD services, scheduling, VM, IPC and device drivers.
Hardware interface	HAL modules expose standard interfaces to vendor hardware implementation.	IOKit and system driver frameworks connect services to device hardware.
Native/system libraries	Bionic libc, media, graphics, SQLite, crypto and native daemons.	libSystem, Core Foundation, graphics/media and security frameworks.
Runtime	ART executes DEX; manages heap, GC, JIT/AOT.	Objective-C runtime supports dynamic dispatch; Swift/Obj-C code is native with ARC.
Framework	Activity, Service, BroadcastReceiver, ContentProvider and system services.	Cocoa Touch/UIKit, UIApplication, UIViewController, notification and background-task APIs.
Application	App process with UID sandbox and lifecycle.	Signed app bundle in per-app sandbox with entitlements.

1.2 PROCESS MODEL AND IPC

Android process creation commonly uses Zygote. The system asks Zygote to fork an application process, then applies the application identity and security context. Framework services communicate through Binder IPC. Binder supports structured transactions and identity propagation; framework APIs hide many low-level details from application code.

iOS process launch is coordinated by launchd and system launch services. XPC and Mach ports provide inter-process communication for permitted system and application services. App processes remain sandboxed and are governed by entitlements and lifecycle rules.

ANDROID COMMANDS FOR LIVE EVIDENCE

```
# Identify connected device
adb devices

# Device and Android version
adb shell getprop ro.build.version.release
adb shell getprop ro.build.version.sdk

# Running application processes
adb shell ps -A | grep your.package.name

# Process status and memory information
adb shell dumpsys meminfo your.package.name

# Activity and process state
adb shell dumpsys activity processes
adb shell dumpsys activity activities

# System services and Binder-related information
adb shell service list
adb shell dumpsys activity | head -n 80

# Application package and permissions
adb shell dumpsys package your.package.name
```

Interpretation: ps confirms whether a process exists; dumpsys meminfo reports memory categories such as PSS and heap allocation; dumpsys activity processes reports process importance and state; package information shows declared/granted permissions. Capture complete output and highlight the lines for the test package. Do not paste only a cropped final number.

1.3 PROCESS LIFECYCLE TEST MATRIX

Condition	Prediction	Evidence to collect	Interpretation
Foreground app	Process remains important; app should remain alive under normal memory conditions.	Activity lifecycle log, ps, dumpsys activity processes.	Foreground importance protects user-visible work but does not guarantee survival under every failure.
Background for five minutes	Process may remain cached; callbacks depend on component and OS policy.	onPause/onStop logs, process state, notification/job records.	A stopped activity is not the same as a dead process.
Low memory / trim	System may trim caches or kill a less-important process.	onTrimMemory logs where delivered, dumpsys meminfo, process list before/after.	Process death may occur without a final callback; durable state must already be saved.

TEST APP LIFECYCLE LOGGING EXAMPLE (KOTLIN)

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.i("LifeTrace", "onCreate restored=" +
            (savedInstanceState != null))
        setContentView(R.layout.activity_main)
    }

    override fun onStart() {
        super.onStart()
        Log.i("LifeTrace", "onStart")
    }

    override fun onResume() {
        super.onResume()
        Log.i("LifeTrace", "onResume")
    }

    override fun onPause() {
        Log.i("LifeTrace", "onPause")
        super.onPause()
    }

    override fun onStop() {
        Log.i("LifeTrace", "onStop")
        super.onStop()
    }

    override fun onSaveInstanceState(outState: Bundle) {
```

```
        outState.putString("draft", currentDraft)
        Log.i("LifeTrace", "onSaveInstanceState")
        super.onSaveInstanceState(outState)
    }

    override fun onDestroy() {
        Log.i("LifeTrace", "onDestroy")
        super.onDestroy()
    }
}
```

1.3 EXECUTION AND LOG COLLECTION

```
# Start log capture
adb logcat -c
adb logcat -s LifeTrace:I ActivityTaskManager:I

# Send app to background
adb shell input keyevent KEYCODE_HOME

# Inspect process
adb shell ps -A | grep your.package.name

# Request a memory trim (test environment only)
adb shell am send-trim-memory your.package.name RUNNING_LOW

# Inspect memory
adb shell dumpsys meminfo your.package.name

# Return to app using launcher or monkey
adb shell monkey -p your.package.name 1
```

Illustrative output only – replace with actual logcat output:

```
I/LifeTrace: onCreate restored=false
I/LifeTrace: onStart
I/LifeTrace: onResume
I/LifeTrace: onPause
I/LifeTrace: onSaveInstanceState
I/LifeTrace: onStop
# After return, callbacks depend on whether process survived:
I/LifeTrace: onStart
I/LifeTrace: onResume
```

The sample is not proof of a real run. A real test must include timestamps, device API level and the process state before and after each condition.

1.4 BOUNDARY TEST

Boundary case: rotate the screen while a form dialog is open and then force process death before returning. This tests whether transient UI state is incorrectly stored only in an Activity field. Expected behaviour: configuration change recreates the Activity; saved state or a ViewModel restores the form draft. If the process is killed, durable state should be restored from saved state or a database. A dialog should not be assumed to survive as a live object; its logical state should be reconstructed.

PART II – TASK 2: ARCHITECTURE ASSUMPTIONS VS MEASURED REALITY

The experiment compares a single-threaded CPU-heavy operation with a background-executor implementation. The same deterministic workload and input must be used for both solutions. The purpose is to measure responsiveness and resource costs, not merely to demonstrate that a second implementation exists.

2.1 APPLICATION ARCHITECTURE

```
UI (Activity / Fragment)
  ↓ user action
ViewModel (state + lifecycle)
  ↓
Use-case / domain operation
  ↓
Repository / data source
  ↓
ExecutorService or coroutine dispatcher
  ↓
CPU-bound worker task
```

The UI thread handles input and rendering. The background executor performs expensive parsing or image processing. Results are returned to the ViewModel and emitted as observable state. The UI updates only when the result is ready.

SINGLE-THREAD VS BACKGROUND IMPLEMENTATION (KOTLIN)

```
fun heavyWork(input: Int): Long {
    var total = 0L
    for (i in 1..input) {
        total += (i.toLong() * i) % 997
    }
    return total
}

// Solution A: main-thread execution (demonstration only)
fun runOnMain() {
    val start = SystemClock.elapsedRealtimeNanos()
    val result = heavyWork(30_000_000)
    val elapsed = (SystemClock.elapsedRealtimeNanos() - start) / 1e6
    Log.i("Bench", "main result=$result ms=$elapsed")
}

// Solution B: background executor
private val executor = Executors.newSingleThreadExecutor()
fun runInBackground() {
    val start = SystemClock.elapsedRealtimeNanos()
    executor.execute {
        val result = heavyWork(30_000_000)
        val elapsed =
            (SystemClock.elapsedRealtimeNanos() - start) / 1e6
        runOnUiThread {
```

```
            Log.i("Bench", "worker result=$result ms=$elapsed")
        }
    }
}
```

For a production application, the main-thread version should not be used for a large workload because it can block input and frame rendering. The executor version keeps the UI responsive, although it may not reduce total compute time on every device.

2.2 COUNTER-EXAMPLE: MORE THREADS CAN BE SLOWER

Use a very small workload, such as summing 100 integers, and compare direct execution with creating a new thread for every invocation. Thread creation, scheduling and synchronisation overhead can exceed the useful computation. Another counter-example is a shared counter protected by one lock: multiple threads compete for the same lock and spend time waiting.

```
fun tinyTask(): Long {
    var value = 0L
    for (i in 1..100) value += i
    return value
}

fun measureDirect(): Long {
    val start = System.nanoTime()
    repeat(10000) { tinyTask() }
    return System.nanoTime() - start
}

fun measureThreaded(): Long {
    val start = System.nanoTime()
    val workers = (1..10000).map {
        Thread { tinyTask() }.apply { start() }
    }
    workers.forEach { it.join() }
    return System.nanoTime() - start
}
```

The expected counter-example is that the threaded implementation is slower for tiny tasks because 10,000 thread creations are disproportionate to the work. This is an experiment hypothesis, not a claimed measurement. Use a bounded thread pool for realistic production work.

2.3 ENGINEERING COMPARISON

Metric	Collection method	Why it matters
Elapsed task time	System.nanoTime / elapsedRealtimeNanos around the same workload.	Measures completion time but not the full cost.
CPU time and utilisation	Android Studio CPU Profiler or Perfetto.	Shows whether parallelism uses more CPU or merely waits.
Thread count/context switches	Profiler trace, top -H where available, Perfetto.	High thread counts and switching can consume energy.
Memory/PSS/heap	dumpsys meminfo and Android Studio Memory Profiler.	Thread stacks and queued objects increase memory.
Frame quality	Frame timing/JankStats/Perfetto frame timeline.	A fast task is not useful if the UI becomes unresponsive.
Worst-case predictability	Repeat tests and report median, p95 and maximum.	Real-time tracking needs bounded delays, not only a good average.

2.4 CHANGED REQUIREMENT: FULL WORKING DAY

If the application must run for a full working day and slightly slower processing is acceptable, the strategy should prioritise energy efficiency over peak throughput. Use a small bounded executor, avoid creating threads per task, batch parsing and uploads, defer non-urgent work through WorkManager, and prefer efficiency cores when the platform scheduler can select them. Keep the UI thread free, but do not use excessive parallelism.

A heterogeneous CPU has performance and efficiency cores. The OS scheduler uses workload characteristics, CPU capacity and thermal/power policy to place work. Applications should not assume they can directly force a specific core on ordinary Android devices. They can reduce urgency, batch work and avoid unnecessary wakeups so the scheduler can make an energy-aware decision.

RESULT RECORDING TEMPLATE

Run	Implementation	Task time	CPU time	Peak PSS	Threads	Janky frames	Energy estimate
1	Main thread	Fill actual	Fill actual	Fill actual	Fill actual	Fill actual	Fill actual
2	Background executor	Fill actual	Fill actual	Fill actual	Fill actual	Fill actual	Fill actual
3	Tiny direct task	Fill actual	Fill actual	Fill actual	Fill actual	Fill actual	Fill actual
4	Tiny threaded task	Fill actual	Fill actual	Fill actual	Fill actual	Fill actual	Fill actual

Do not invent profiler numbers. Attach screenshots of the CPU profiler/Perfetto trace and explain any mismatch between predicted and measured results.

PART II – TASK 3: RUNTIME BEHAVIOUR UNDER CHANGING CONDITIONS

The benchmark contains three workloads: CPU-bound calculation, allocation-heavy object creation and UI-list rendering. It records cold and warm launch times, ART compilation state, GC events and heap growth. The failure stage deliberately introduces a memory leak and a main-thread blocking call in a test-only build.

3.1 BENCHMARK WORKLOADS

- CPU-bound: deterministic loop or sorting workload with fixed input.
- Allocation-heavy: create short-lived objects and retain a controlled subset to observe heap growth and GC.
- UI-list rendering: display a large list with repeated row binding and scrolling; compare first render and subsequent scroll frames.

3.2 COMPILATION AND STARTUP COMMANDS

```
# Package compile status (command availability varies by Android version)
adb shell dumpsys package your.package.name

# Inspect package compilation state
adb shell cmd package compile --help

# Example compilation filters; use only those supported by the device
adb shell cmd package compile -m verify -f your.package.name
adb shell cmd package compile -m speed-profile -f your.package.name
adb shell cmd package compile -m speed -f your.package.name

# Clear app data only when safe for the test
adb shell pm clear your.package.name

# Capture startup-related logs
adb logcat -c
adb logcat -v time | grep -E "ActivityTaskManager|art|GC|your.package.name"
```

Compilation filters and exact command support vary with Android release and device build. Record the command response and Android version. Do not assume that a command accepted on one emulator is accepted on every device.

3.3 GC AND HEAP OBSERVATION

```
# App memory summary
adb shell dumpsys meminfo your.package.name

# Capture runtime log messages
adb logcat -v time | grep -iE "gc|art|heap|alloc"

# Optional: open Android Studio Profiler
# Memory Profiler → record allocations → force GC → inspect heap
```

Interpretation guide: increasing allocated heap does not by itself prove a leak. A leak is suspected when retained objects and post-GC live heap continue increasing across repeated equivalent operations. GC events show collection frequency and pause duration. Frequent collections with little reclaimed memory may indicate high allocation churn or retained references.

3.4 CONTROLLED MEMORY-LEAK EXAMPLE

```
object TestLeak {
    // Test-only: intentionally retains every Activity reference.
    val leakedActivities = mutableListOf<Activity>()

    fun add(activity: Activity) {
        leakedActivities.add(activity)
    }
}

// In a test Activity only:
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    TestLeak.add(this) // Deliberate leak for experiment
}
```

Detection: repeat Activity creation/destruction, force garbage collection through the profiler, and inspect retained Activity instances. A correct leak detector reports a path from a GC root (the static list) to the destroyed Activity. Remove this test code immediately after the experiment.

3.5 MAIN-THREAD BLOCKING FAILURE

```
// Test-only anti-pattern: blocks UI thread.
Thread.sleep(5000)

// Production correction:
lifecycleScope.launch {
    val result = withContext(Dispatchers.IO) {
        repository.loadData()
    }
    render(result)
}
```

The Android input dispatch watchdog may report an Application Not Responding (ANR) if the main thread fails to process input within the system's timeout. StrictMode can detect selected disk/network operations on the main thread. The runtime heap manager and garbage collector report memory pressure; the OS process manager may eventually terminate the process.

3.6 CROSS-PLATFORM RUNTIME COMPARISON

Runtime	Compilation/execution	Memory model	UI-thread and native access	Likely trade-off
Android ART	DEX with interpretation, JIT and profile-guided AOT.	Managed heap with tracing GC; JNI/native memory separate.	Android framework on main thread; JNI/NDK for native APIs.	Strong Android integration; GC/allocation and ABI choices affect performance.
iOS Swift/Objective-C	Native machine code; Objective-C runtime handles dynamic dispatch.	ARC reference counting; cycles can leak.	UIKit/SwiftUI main-thread rules; direct system framework calls.	Native integration and predictable ARC operations; careful cycle management required.
Flutter	Dart AOT for release builds; Flutter engine renders UI.	Dart managed heap and garbage collection.	Dart UI/isolate model; platform channels/plugins access native APIs.	Consistent cross-platform UI; engine adds memory footprint and platform-channel costs.
React Native	JavaScript engine executes JS; native components accessed through framework/bridge or newer architecture.	JS garbage collection plus native object lifecycle.	JS thread and native UI/runtime coordination.	Shared business logic; bridge/serialization and JS scheduling can affect latency.
WebView hybrid	HTML/CSS/JS in browser engine.	JavaScript GC and browser-managed resources.	Web APIs plus native bridge for device features.	High reuse and web skills; less direct control over native performance/background work.

Recommendation: for a healthcare or delivery application where background execution, security and native services are critical, native platform implementations provide the clearest lifecycle and security integration. If a single codebase is a strict business requirement, Flutter is a reasonable candidate for a controlled evaluation, but its memory footprint, background plugins and native API integration must be benchmarked on the target devices. The recommendation must be validated against measured Stage A–C evidence.

PART II – TASK 4: SECURE MOBILE APPLICATION AUDIT

The audit accepts requested permissions, data stores, network endpoints and exported components. It maps each asset to threats, controls and residual risk. STRIDE is used: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service and Elevation of privilege.

4.1 INPUT MODEL

Input category	Example fields	Audit question
Permissions	CAMERA, ACCESS_FINE_LOCATION, INTERNET	Are permissions necessary and requested at the correct time?
Data stores	SharedPreferences, SQLite, files, Keystore	Is sensitive data private, encrypted and recoverable?
Network	HTTPS API endpoint, TLS version, certificate policy	Is server identity verified and is traffic encrypted?
Components	Activities, services, receivers, providers, deep links	Can an untrusted caller invoke an internal component?
Signing	Package signing identity and update channel	Can an attacker replace the application?

4.2 STRIDE THREAT REGISTER

Threat	Asset / attack path	Protection mechanism	Residual risk
Spoofing	Attacker steals or reuses a bearer token.	Short-lived tokens, secure storage, server-side session revocation, MFA where appropriate.	Compromised unlocked device may expose an active session.
Tampering	Attacker modifies local database or API request.	Authenticated encryption, integrity checks, TLS, server validation.	Rooted OS or compromised process may alter data before encryption.
Repudiation	Worker denies submitting a patient record.	Server audit log, authenticated user, timestamp, request ID and acknowledgement.	Shared credentials or inaccurate device time weaken attribution.
Information disclosure	Patient data read from storage or intercepted on network.	App sandbox, encrypted database/files, Keystore/Keychain, TLS.	Plaintext may exist briefly in app memory or screenshots.
Denial of service	Malformed input, storage exhaustion or repeated requests.	Input limits, rate limiting, bounded queues, retry backoff and storage monitoring.	Network outage or device resource exhaustion can still interrupt service.
Elevation of privilege	Exported component or overbroad	Non-exported components by	Vulnerable trusted library or OS

	permission exposes privileged operation.	default, signature permissions, runtime permission checks, SELinux/sandbox.	compromise may bypass app-level assumptions.

4.3 AUDIT DECISION TRACE

Asset	Attack vector	Mechanism applied	Verdict
Auth token	Read from another app or backup.	Private app storage + Keystore-protected encryption + no backup of secrets.	Blocked in normal sandbox; partially exposed on compromised/rooted device.
Local database	Copy database file from shared storage.	Encrypted database, app-private path, key access control.	Blocked for ordinary apps; residual risk while app is unlocked.
API traffic	Wi-Fi interception or response modification.	TLS with hostname validation and server authentication.	Blocked against passive interception; certificate or endpoint compromise remains.
Exported Activity	Untrusted app launches internal screen with crafted extras.	android:exported=false unless required; validate deep links and caller input.	Blocked if not exported; exposed if intentionally exported without checks.

Verdict logic: Blocked means the named control prevents the tested attack under stated assumptions. Partially Blocked means the control raises the cost but leaves a meaningful residual path. Exposed means the test can access or modify the asset. A verdict must name the tested device state and OS version.

4.4 PREDICTION BEFORE WEAKENING A CONTROL

Selected control: allow cleartext traffic. Prediction: API request bodies and authentication material sent over HTTP become visible to an observer on the same network path. The audit should change the API traffic verdict from Blocked to Exposed or Partially Blocked. A controlled test can use a local test endpoint and packet capture; never intercept third-party traffic.

4.5 THIRD-PARTY LIBRARY INCIDENT

Asset affected: device identifier and associated user/device metadata. The OS sandbox cannot prevent an application library from reading data that the host application itself has permission to access and then transmitting it. Permission controls limit access to protected identifiers, but they do not guarantee that an authorised app will use data safely. Detection points include static analysis of network destinations and cleartext use, runtime network hooks, and packet capture in a controlled test. Remediation: remove or update the library, stop collecting unnecessary identifiers, route all network traffic through an approved TLS client, apply data minimisation and review the library's permissions and privacy behaviour.

4.6 BIOMETRICS AND ENTERPRISE WORK PROFILE

Biometric login changes the threat model by adding an authentication factor and a hardware-backed trust decision. The app should use platform biometric APIs and bind key use to user authentication where supported; it should not store biometric templates. Key attestation may help a server evaluate whether keys are hardware-backed and generated in an acceptable environment.

An enterprise work profile introduces a second data boundary. Corporate data must remain in the managed profile/container and must not be copied to personal apps. The audit must test cross-profile intents, clipboard, file sharing, backup, notifications, screenshots and remote wipe. Device management policy should define what is wiped: work data only or the entire device, depending on ownership and legal policy.

PART II – TASK 5: THREE POWER ENGINEERS, ONE DEVICE

The experiment compares CPU DVFS, OS idle controls and application-level batching. Each technique must be tested with the same application and device profile. Battery percentage alone is insufficient because it does not identify which component caused the energy change.

5.1 TECHNIQUE 1 – DVFS AND CORE SELECTION

Dynamic Voltage and Frequency Scaling changes CPU operating frequency and voltage according to load, thermal state and power policy. Higher frequency can complete a task sooner but may require higher voltage and power. Efficiency cores can perform suitable workloads with lower energy, while performance cores are useful for short latency-sensitive bursts. Ordinary apps generally do not control exact core placement; the OS scheduler makes placement decisions.

5.2 TECHNIQUE 2 – DOZE AND APP STANDBY

Doze reduces background activity when the device is stationary, unplugged and idle. App Standby Buckets classify app activity and influence job/alarm/network allowances. Background execution limits prevent an application from continuously running arbitrary work. These mechanisms are triggered by device state and usage, not by a developer's preference alone.

5.3 TECHNIQUE 3 – APP-LEVEL BATCHING

- Use sensor batching to collect several samples before waking the application where supported.
- Coalesce network requests so one radio-active period carries multiple records.
- Use WorkManager/JobScheduler for deferrable work and specify constraints such as network and charging.
- Release wake locks in finally blocks and apply explicit timeouts.
- Use exponential backoff with jitter after network failure to avoid repeated wakeups.

POWER INSTRUMENTATION COMMANDS

```
# Reset battery statistics before a controlled run
adb shell dumpsys batterystats --reset

# Inspect battery statistics after the run
adb shell dumpsys batterystats

# Request a device idle mode for a test device
adb shell dumpsys deviceidle force-idle

# Exit forced idle
adb shell dumpsys deviceidle unforce
adb shell dumpsys deviceidle step

# Inspect jobs and alarms
adb shell dumpsys jobscheduler
adb shell dumpsys alarm

# Inspect wakelocks in battery statistics
adb shell dumpsys batterystats | grep -i wake
```

Use these commands only on a test device/emulator. Exact output and supported commands vary by Android version. Battery Historian or Android Studio Energy Profiler can provide a more interpretable timeline. Record screen state, charging state, network type, GPS state, run duration and workload for each test.

MEASUREMENT PLAN

Run	Technique enabled	Trigger	Measurements	Side-effect
A	DVFS / default scheduler	CPU load and thermal/power policy	CPU frequency where exposed, CPU time, estimated energy, temperature	Task may take longer on efficiency cores.
B	Doze / App Standby	Screen off, stationary, unplugged	Wakeups, jobs deferred, radio-active time, delivery latency	Background updates delayed.
C	Batching / WorkManager	Queue reaches batch size or job constraint met	Wake lock duration, radio wakeups, batch size, estimated mAh	Data upload is less immediate.
D	Conflict case	Hold partial wake lock while device enters idle	Idle transition, wake-lock duration, job timing	Wake lock can undermine idle savings.

STRESS CASE – CROSS-LAYER CONFLICT

A partial wake lock held continuously by the application can prevent the CPU from entering a deep sleep state. In that case, the OS-level idle policy cannot deliver its expected benefit because the application is actively requesting that the processor remain awake. Similarly, frequent exact alarms can repeatedly wake the device and defeat batching. This demonstrates that a power-saving mechanism at one layer can be defeated by behaviour at another layer.

Refinement: remove the permanent wake lock; use a short bounded lock only around critical processing; move routine work to scheduled jobs; batch sensor and network operations; and record wake-lock acquisition/release timestamps. A test should show that the conflicting build has more wake time and that the refined build permits longer idle periods.

INDUSTRY DECISION

Environment	Important characteristics	Recommended emphasis
A – Always-on location/sensor tracking	Sensor continuity, GPS/radio energy, user-visible ongoing work and latency.	Use efficient sensor batching, adaptive sampling and a foreground service only during active tracking; upload in batches.
B – Media/gaming-intensive	Sustained CPU/GPU load, display power, thermal throttling and frame deadlines.	Prioritise frame pacing and thermal stability; reduce unnecessary background work; accept controlled performance limits under heat.
C – Managed device fleet	OS-version diversity, enterprise policies, remote management, predictable scheduling and support cost.	Use WorkManager/JobScheduler constraints, policy-based configuration, fleet telemetry and conservative compatibility rules.

The appropriate policy depends on the dominant hardware cost. GPS and radio may dominate location tracking; GPU/display may dominate gaming; repeated wakeups and management overhead matter across large fleets. No single technique is sufficient in every environment.

PART II – TASK 6: BATTERY-AWARE, SECURE FIELD-TRACKING PROTOTYPE

The prototype models a delivery phone that samples location/sensor data, stores records locally and uploads them securely. The objective is to maximise useful tracking data delivered per unit of energy while protecting the payload.

6.1 PROBLEM MODEL

Element	Definition
Input	Battery percentage and charging state, network type, motion state, temperature, queued records, last upload time and emergency flag.
Output	Sampling interval, upload action, queue status, encryption status and estimated energy per cycle.
Objective	Deliver useful and timely tracking records while minimising unnecessary CPU, GPS and radio wakeups.
Constraints	Full-shift battery, intermittent network, secure local storage, authenticated upload, acceptable emergency latency and bounded queue size.

Battery percentage alone is insufficient. Two devices at 20% may require different actions: one may be charging on Wi-Fi and safely upload a large batch; another may be stationary on weak LTE with a hot battery and should defer non-urgent upload. Network type, charging, motion, temperature and queue age change the decision.

6.2 POLICY STATE MODEL

Policy[BatteryState][NetworkState][MotionState] → {SamplingRate, UploadAction}

BatteryState = NORMAL | LOW | CRITICAL | CHARGING
NetworkState = WIFI | CELLULAR | OFFLINE
MotionState = MOVING | STATIONARY

Every state transition:
1. Validate current state and event timestamp.
2. Select sampling interval and upload policy.
3. Encrypt each record before queue persistence.
4. Upload only over authenticated TLS.
5. Mark acknowledged records only after server acknowledgement.

6.2 LAYERED ARCHITECTURE

Presentation layer (main thread)
- status, map, battery and sync indicators

↓

Domain layer (background executor)
- policy evaluator
- sampling controller
- sync state machine

↓

Data layer
- encrypted queue / database
- record UUID and acknowledgement journal

↓

OS services
- location and sensor APIs
- foreground service during active tracking
- WorkManager for deferred uploads
- Keystore for encryption key

↓

Network
- TLS authenticated API

↓

Fleet backend / idempotent ingestion

The policy evaluator is a pure function so it can be tested independently. UI rendering does not run on the background executor. Location sampling is controlled by a foreground service during an explicitly active tracking session. Deferred upload is scheduled through WorkManager and respects network/charging constraints.

6.3 PROTOTYPE SIMULATION – PYTHON

```
from dataclasses import dataclass

@dataclass
class Condition:
    name: str
    battery: int
    charging: bool
    network: str       # WIFI, CELLULAR, OFFLINE
    motion: str        # MOVING, STATIONARY
    hot: bool = False
    emergency: bool = False

def choose_policy(c):
    if c.emergency:
        return 5, "UPLOAD_NOW_HIGH_PRIORITY"
    if c.hot:
        return 120, "DEFER_UNTIL_COOL_OR_WIFI"
    if c.charging and c.network == "WIFI":
        return 10, "UPLOAD_BATCH_NOW"
    if c.network == "OFFLINE":
```

```
        return (30 if c.motion == "MOVING" else 120), "QUEUE_ONLY"
    if c.battery <= 15:
        return 180, "DEFER_UNTIL_CHARGING_OR_WIFI"
    if c.battery <= 25 and c.motion == "STATIONARY":
        return 120, "DEFER_UNTIL_WIFI"
    if c.network == "CELLULAR" and c.battery < 50:
        return 60, "UPLOAD_SMALL_BATCH"
    return (15 if c.motion == "MOVING" else 60), "BATCH_UPLOAD"

def estimate_energy(interval_seconds, upload_action):
    # Illustrative model only; replace with measured energy coefficients.
    gps = 0.018 * (60 / max(interval_seconds, 1))
    cpu = 0.004
    radio = 0.025 if "UPLOAD" in upload_action else 0.003
    return round(gps + cpu + radio, 4)

scenarios = [
    Condition("Charging Wi-Fi", 80, True, "WIFI", "MOVING"),
    Condition("80% LTE moving", 80, False, "CELLULAR", "MOVING"),
    Condition("20% stationary LTE", 20, False, "CELLULAR", "STATIONARY"),
    Condition("Low power 12%", 12, False, "CELLULAR", "MOVING"),
    Condition("No connectivity", 55, False, "OFFLINE", "MOVING"),
    Condition("Device hot", 45, False, "WIFI", "MOVING", hot=True),
    Condition("Emergency", 18, False, "CELLULAR", "MOVING", emergency=True)
]

for c in scenarios:
    interval, action = choose_policy(c)
    energy = estimate_energy(interval, action)
    print(c.name, interval, action, energy,
          "AES-GCM before queue; TLS for upload")
```

6.3 SCENARIO RESULTS – ILLUSTRATIVE SIMULATION

The following values are generated by the simple illustrative energy formula in the code. They are not measured mAh values and must not be reported as hardware measurements. The formula is only a transparent way to demonstrate that the policy selects a sampling interval and upload decision for each state.

Condition	Sampling interval	Upload decision	Protection	Estimated model units
Charging Wi-Fi	10 s	Upload batch now	AES-GCM before queue; TLS	0.1370
80% LTE moving	15 s	Batch upload	AES-GCM + TLS	0.1290
20% stationary LTE	120 s	Defer until Wi-Fi	AES-GCM before queue	0.0160
Low power 12%	180 s	Defer until charging/Wi-Fi	AES-GCM before queue	0.0130
No connectivity	30 s	Queue only	AES-GCM before queue	0.0430
Device hot	120 s	Defer until cool or Wi-Fi	AES-GCM before queue	0.0160
Emergency	5 s	Upload now, high priority	AES-GCM + TLS	0.2450

The energy model uses arbitrary coefficients to illustrate policy differences. A real experiment must derive energy estimates from Battery Historian, device power profiles, an external power monitor or a documented measurement method. If no reliable mAh estimate is available, report wakeup counts, GPS active duration, radio-active duration and relative energy indicators instead.

6.4 VALIDATION AGAINST ALWAYS-ON BASELINE

For hand validation, select three conditions: 80% battery on LTE while moving; 20% battery while stationary on LTE; and offline while moving. The expected policy outputs are shown below. The baseline samples continuously at a fixed 5-second interval and attempts immediate upload, producing repeated radio wakeups when network is available and failed retries when offline.

Condition	Adaptive expected	Always-on baseline	Validation conclusion
80% LTE moving	15 s sample; batch upload.	5 s sample; immediate upload per cycle.	Adaptive policy reduces sampling and radio activations; may increase delivery latency.
20% stationary LTE	120 s sample; defer until Wi-Fi.	5 s sample; repeated LTE uploads.	Adaptive policy reduces nonessential work; acceptable if stationary updates are not urgent.
Offline moving	30 s sample; queue only.	5 s sample plus failed upload attempts.	Adaptive policy avoids futile network wakeups while retaining movement samples.

Independent validation increases confidence because the expected result is derived separately from the implementation. If the prototype output differs from the hand calculation, the policy branch, state input or boundary condition is wrong and can be corrected before deployment.

6.5 SCALABILITY AND LOAD

Load	Policy evaluations	Wake-ups	Memory / battery observation
100 records	Measure actual	Measure actual	Small queue; inspect fixed per-wakeup overhead.
1,000 records	Measure actual	Measure actual	Batching should reduce repeated radio activations.
10,000 records	Measure actual	Measure actual	Check database indexing, queue paging, memory bounds and upload batch size.
1 / 5 / 10 / 20 devices	Measure actual	Measure actual	Check server ingestion throughput, retry storms and fleet scheduling.

Batching eliminates repeated radio wakeups because several records share one radio-active period. However, continuous high-frequency tracking remains expensive because GPS/sensor sampling and CPU processing continue to occur, and the volume of collected data grows. Batching reduces fixed connection overhead but does not eliminate the energy cost of acquiring every sample.

6.6 BUSINESS REQUIREMENT CHANGE: 5,000 DEVICES AND EMERGENCY UPDATES

The existing battery-saving policy becomes unsuitable during emergencies because it deliberately defers uploads and increases sampling intervals. The changed property is the required latency and reliability: updates must arrive within a few seconds, and reliable delivery now has higher priority than battery conservation.

Strategy to investigate: introduce a priority-based policy with an emergency state, a foreground tracking mode where permitted, high-priority push/alert coordination, server-side fleet management and edge/cloud processing. Emergency records should bypass ordinary batch delay, use authenticated TLS and be retried with bounded backoff. The backend must support 5,000 devices, deduplication, rate limiting and priority queues.

Accepted trade-offs: higher battery consumption, increased radio and CPU wakeups, more heat, greater network cost and increased privacy exposure from more frequent location collection. The organisation should define when emergency mode begins and ends, who can activate it, how the event is audited and how collection returns to normal policy.

6.7 TECHNICAL REFLECTION (UNDER 150 WORDS)

A power-saving technique is effective only under the conditions for which its assumptions hold. Batching reduces repeated radio wakeups when updates can wait, but it is unsuitable when an emergency requires a location update within seconds. Doze can save energy while a device is stationary, but a permanent wake lock or repeated exact alarms can prevent deep idle. Lower sampling frequency saves energy but may lose useful movement detail. Therefore, no single mobile OS feature or policy is optimal for every application. The correct design combines battery, network, motion, temperature, queue age and urgency state, then selects a policy that satisfies the current requirement. Measurements must confirm that the selected policy improves energy use without violating the application's latency, accuracy or security needs.

OVERALL CONCLUSION

This assignment demonstrates that mobile computing is an interaction among hardware, kernel, runtime, framework, application lifecycle, security controls and power-management policy. A reliable architecture identifies the boundary between each layer and the mechanism used to cross it. Android, iOS and Windows share broad goals but differ in process creation, runtime, IPC, sandboxing and background execution. These differences directly affect messaging, sensor collection and application responsiveness.

The design tasks show that security must be applied at several layers: least-privilege permissions, sandbox isolation, protected keys, signed packages, authenticated TLS and server-side authorisation. Power efficiency similarly requires cooperation among the kernel, OS services and application. A permanent wake lock or high-frequency polling can defeat system-level idle mechanisms. For offline field systems, encrypted durable queues, stable record identifiers and idempotent server acknowledgements prevent loss and duplication.

The practical section provides a reproducible path for measuring process state, lifecycle, runtime compilation, garbage collection, memory, security decisions and power behaviour. Device-specific values must be captured on the actual test environment. The most important engineering lesson is to design, instrument, test, break, debug, validate and defend the system rather than stopping when an application launches.